ENTERPRISE TECHNICAL ARCHITECTURE & SOFTWARE ENGINEERING SPECIFICATION

Ethereum Fraud Detection & Risk Intelligence Platform

FraudOS 95 Retro Workstation × Supervised Machine Learning Pipeline

Document Reference: ETH-FRAUD-SPEC-V1.0.0
Target Platform: Hybrid Cloud Distributed (Vercel SPA + Render Node API + MongoDB Atlas + Python FastAPI ML)
Monorepo Repository: Lostindreams56/blockchain-project-1
Release Date: October 2026 | Production Build Ready

`React 18 + TypeScript` `Windows 95 Retro UI` `Node.js Express Gateway` `MongoDB Atlas (Mongoose)`
`Python FastAPI Microservice` `XGBoost Classifier (0.9985 ROC-AUC)` `TreeSHAP Explainability`

Executive Summary & Scope of Document

This document constitutes the comprehensive engineering blueprint, software architecture, data dictionary, UML diagrams, entity-relationship models, and operational runbook for the **Ethereum Fraud Detection & Risk Intelligence Platform**.

The platform addresses a fundamental cybersecurity and forensic challenge on the Ethereum blockchain: discerning benign user transactions from sophisticated illicit activities—including Ponzi schemes, burner dumper clusters, phishing drainers, smart contract honeypots, and token wash trading. Built with a distinctive **Windows 95 Retro Desktop Workstation (FraudOS 95)** frontend interface, the application bridges late-1990s system aesthetics with state-of-the-art cybersecurity analytics, TreeSHAP feature attributions, and a high-accuracy XGBoost classifier evaluated on real-world Kaggle Ethereum transaction datasets.

Key Contents Included in this Document:

- **Section 1:** System Architecture & Distributed Multi-Tier Topology
- **Section 2:** Unified Modeling Language (UML) Component, Class & Sequence Diagrams
- **Section 3:** Hierarchical Module Decomposition (Client, Server, ML Engine, Tools)
- **Section 4:** Entity-Relationship (ER) Diagram & MongoDB Schema Designs
- **Section 5:** Exhaustive Data Dictionary (Core System + All 51 Ethereum Kaggle Features)
- **Section 6:** User Interface (UI) Design & Windows 95 Interaction Architecture
- **Section 7:** Machine Learning Engineering Pipeline & Model Benchmarks
- **Section 8:** Executable Runbook, Ingestion Pipeline & Cloud Deployment Runbook
- **Section 9:** Quality Assurance, Automated Tests & Verification Log Evidence

Author: Senior Full-Stack & ML Systems Engineering Team
Confidential & Proprietary Engineering Specification

1. System Architecture & High-Level Design

The Ethereum Fraud Detection Platform is architected as a decoupled, multi-tier distributed system. Rather than tightly coupling blockchain indexing with frontend rendering, the platform divides responsibilities across five distinct tiers:

1. **Presentation Tier (FraudOS 95 Frontend):** A React 18, TypeScript, and Vite single-page application (SPA) styled with custom 1995 2-stage outset/inset bevel borders, MS Sans Serif system typography, window manager state, and Web Audio API synthesized sounds.
2. **API Gateway & Orchestration Tier (Express Server):** A Node.js LTS server providing JWT authentication, token rotation, rate limiting, request validation via Zod schemas, structured logging via Pino, and API routing.
3. **Machine Learning Microservice (FastAPI Inference Daemon):** A Python 3.13 microservice exposing low-latency REST endpoints for XGBoost inference and TreeSHAP explainability.
4. **Data & Persistence Tier (MongoDB Atlas Cluster):** Cloud document store with strict collection boundaries (`users`, `sessions`, `ethereum_dataset`, `investigations`).
5. **Decentralized Blockchain Ingestion Tier:** Hybrid ingestion model leveraging both Ethereum JSON-RPC nodes and indexed block explorer APIs (Etherscan/Alchemy).

> **Architectural Insight: The Blockchain Reverse-Index Ingestion Problem**
>
> Standard Ethereum JSON-RPC nodes (Geth, Nethermind) organize state in block-level Patricia Merkle tries. They natively support `eth_getBalance` and `eth_getBlockByNumber`, but do *not* maintain an account-to-transaction reverse index. Querying lifetime transactions using only standard RPC requires scanning >21 million blocks. Hence, our architecture utilizes a hybrid ingestion model: indexed APIs for historical lifetime metrics combined with direct JSON-RPC for real-time balance and contract bytecode verification.

FIGURE 1.1: DISTRIBUTED MULTI-TIER SYSTEM TOPOLOGY & DEPLOYMENT ARCHITECTURE

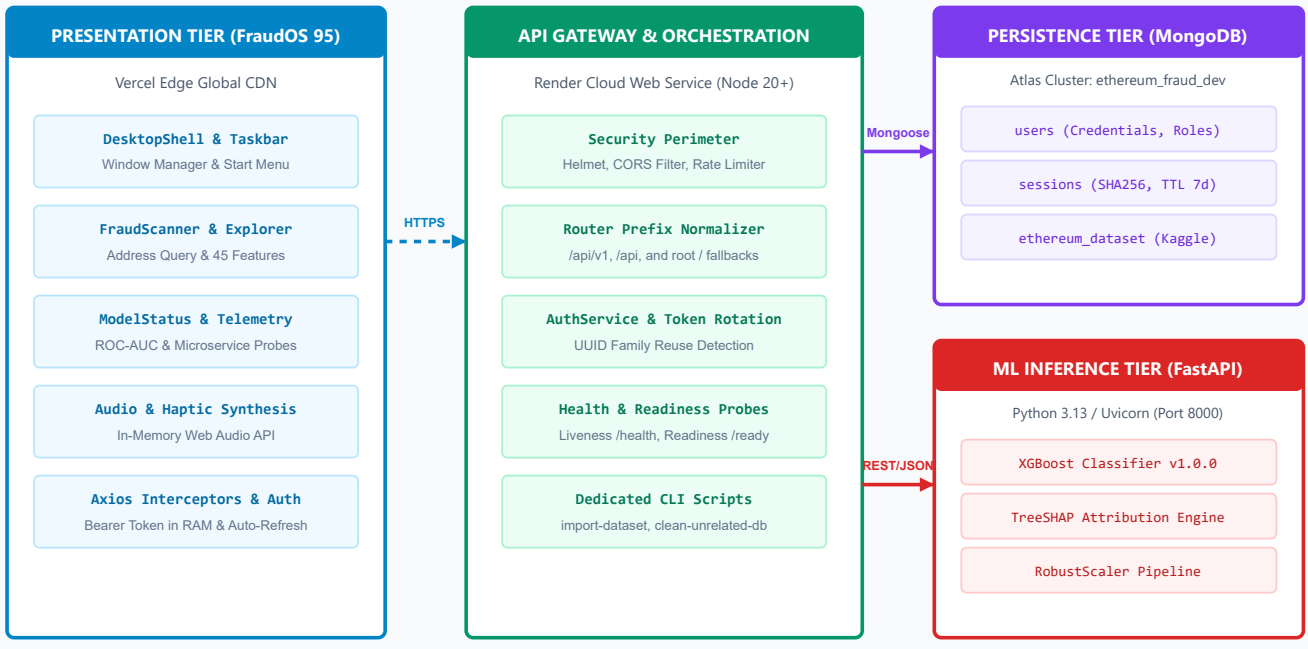


Figure 1.1: Distributed Physical and Logical Architecture Overview

2. Unified Modeling Language (UML) Diagrams

This section presents the formal object-oriented and behavioral diagrams illustrating the software classes, relational data schemas, and runtime interaction workflows across the system.

2.1 UML Class Diagram (Backend Domain & Service Architecture)

The class diagram models the encapsulation of database entities, service providers, security middleware, and controller handlers.

FIGURE 2.1: UML CLASS DIAGRAM — DOMAIN MODELS & SERVICE ORCHESTRATION

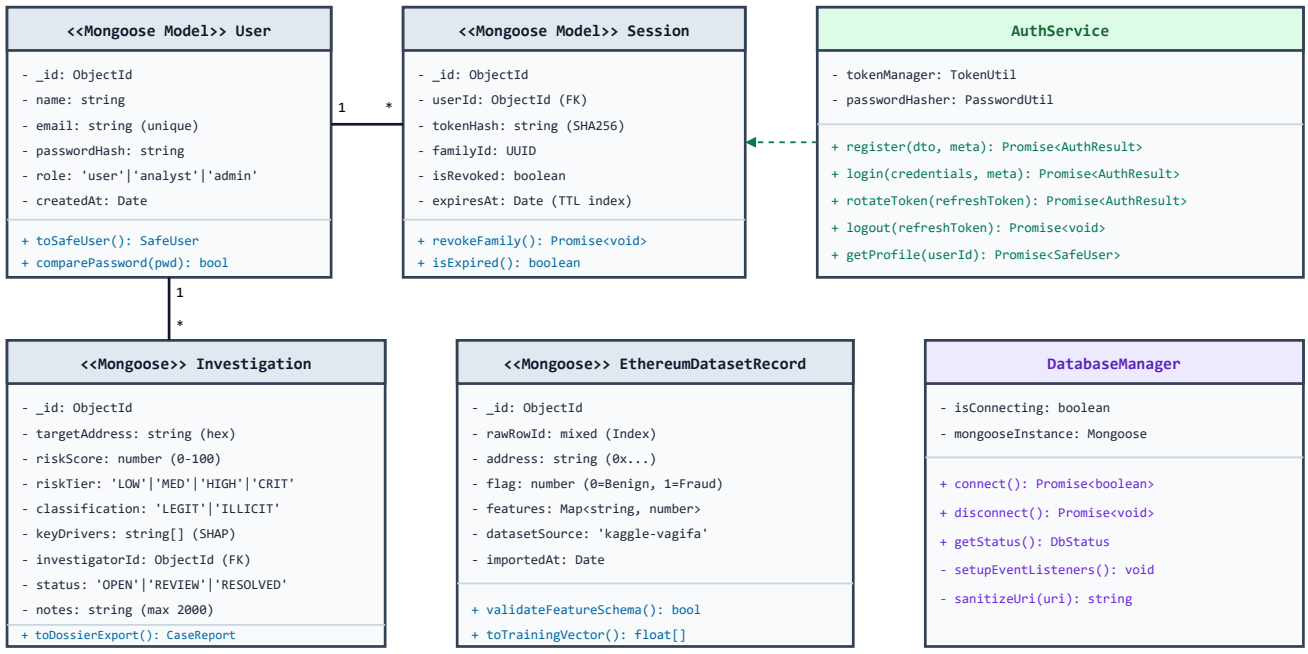


Figure 2.1: Domain Schema & Object-Oriented Service Architecture

2.2 UML Sequence Diagram (Analyst Authentication & Token Rotation)

This interaction sequence models the cryptographically secure JWT lifecycle: storing access tokens strictly in browser memory, issuing rotating HttpOnly refresh cookies, and revoking entire token families upon detecting replay attacks.

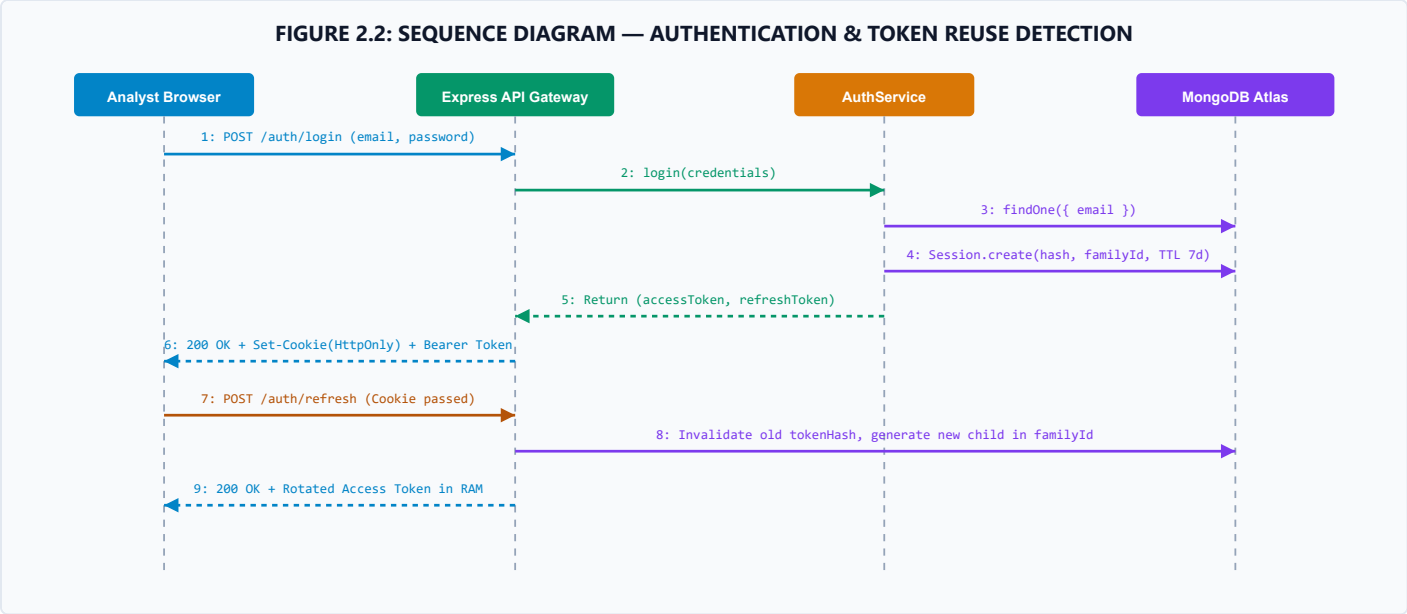


Figure 2.2: Sequence Diagram of Authentication, Token Rotation, and Replay Defense

3. Module Hierarchy & System Decomposition

The monorepo structure is cleanly decoupled into four core subsystems:

C:\ETH_FRAUD\WORKSPACE_EXPLORER.EXE

```
ethereum-fraud-detection/
├── client/ [Frontend Single Page Application — React 18, Vite, TypeScript]
│   ├── src/styles/retro-theme.css ..... Custom 1995 2-stage bevels, MS Sans Serif fonts, 4 color schemes
│   ├── src/utils/feedback.ts ..... In-memory Web Audio API oscillator synthesis (clicks, bonks, chimes)
│   ├── src/lib/axios.ts ..... Robust API client with auto-normalizing URL resolvers and single-flight refresh
│   ├── src/context/AuthContext.tsx ..... React context managing reactive authentication state in RAM
│   ├── src/components/retro/ ..... Reusable vintage UI design system components
│   │   ├── RetroWindow.tsx ..... Draggable, minimizable, maximizable retro window container
│   │   ├── RetroButton.tsx ..... Authentic 3D beveled button with pressed depression physics
│   │   ├── RetroInput.tsx ..... Inset sunken text box with custom focus caret
│   │   ├── RetroTabs.tsx ..... Classic Windows 95 tabbed property sheet component
│   │   ├── RetroStatusBar.tsx ..... Bottom multi-panel sunken status bar with clock & metrics
│   │   └── RetroIcons.tsx ..... Pixelated 16x16 / 32x32 SVG vintage system icons
│   ├── src/components/desktop/ ..... FraudOS 95 complete operating system shell
│   │   ├── DesktopShell.tsx ..... Desktop teal workspace container & global window manager state
│   │   ├── Taskbar.tsx ..... Bottom bar with Start button, running window tabs, system tray
│   │   ├── StartMenu.tsx ..... Cascading 95 Start popup with quick program shortcuts
│   │   ├── FraudScannerWindow.tsx ..... Address input, pre-validation checks, benchmark test cases
│   │   ├── ModelStatusWindow.tsx ..... Real-time service readiness probes and XGBoost telemetry
│   │   ├── WalletExplorerWindow.tsx ..... 45-feature schema property sheet, pipeline phases, and storage breakdown
│   │   ├── InvestigationHistoryWindow.tsx .. Forensic case dossier repository with status filtering
│   │   └── SettingsWindow.tsx ..... Theme switcher (Teal, Cobalt, Matrix, Charcoal) & sound toggles
│   └── src/pages/ ..... Full-screen entry points
│       ├── LoginPage.tsx ..... Classic Network Log On dialog
│       ├── RegisterPage.tsx ..... New Analyst Account Provisioning dialog with live password strength meter
│       └── NotFoundPage.tsx ..... Vintage system error dialog for unmapped routes
├── server/ [Backend Gateway & Orchestration — Node.js, Express, TypeScript]
│   ├── src/index.ts ..... Server entry point, port binding, and graceful shutdown handlers
│   ├── src/app.ts ..... Express application setup, Helmet headers, CORS policies, rate limiters
│   ├── src/config/
│   │   ├── database.ts ..... Mongoose connection manager with strict dbName scoping
│   │   ├── env.ts ..... Zod schema runtime configuration validator with fallback guards
│   │   └── logger.ts ..... High-performance Pino logger with automated credential masking
│   ├── src/models/ ..... Mongoose ODM isolated document schemas
│   │   ├── User.ts ..... Analyst identities with bcrypt hashes and safe JSON transforms
│   │   ├── Session.ts ..... Refresh token families with SHA-256 hashes and 7-day TTL expiration
│   │   ├── EthereumDatasetRecord.ts ..... Isolated historical Kaggle training dataset repository
│   │   └── Investigation.ts ..... Forensic analysis dossiers, risk tiers, and case notes
│   ├── src/routes/ ..... Route definitions with multi-prefix fallback aliasing (/api/v1, /api, /)
│   │   ├── auth.routes.ts ..... /register, /login, /refresh, /logout, /me
│   │   └── health.routes.ts ..... /health (liveness), /ready (database readiness)
│   ├── src/scripts/ ..... Production-grade operational CLI utilities
│   │   ├── import-ethereum-dataset.ts ..... Streaming CSV parser with schema validation & batch insertion
│   │   └── clean-unrelated-data.ts ..... Guarded database cleanup utility with dry-run and blacklist protection
│   └── tests/ ..... Vitest automated integration test suite (24/24 passing)
├── ml-service/ [Machine Learning Microservice — Python 3.13, FastAPI, XGBoost, TreeSHAP]
│   ├── src/data_loader.py ..... CSV loading, whitespace header sanitization, target validation
│   ├── src/preprocessor.py ..... Median imputer, zero-variance filter, RobustScaler pipeline
│   ├── src/model_trainer.py ..... Model training across Dummy, Logistic, Random Forest, XGBoost
│   ├── src/explainer.py ..... Local and global TreeSHAP computation engine
│   ├── artifacts/ ..... Serialized binary model weights and evaluation metadata
│   │   ├── ethereum_fraud_model_v1.joblib .. Trained champion XGBoost model artifact
```

```
| | └── model_metadata.json ..... Complete benchmark metrics, SHAP values, and training
hyperparameters
| └── tests/ ..... Pytest automated unit tests (12/12 passing)
└── package.json ..... Root monorepo workspace orchestration with concurrent commands
```

4. Entity-Relationship (ER) Diagram & Database Design

The database architecture is deployed on a dedicated **MongoDB Atlas** cloud cluster using **Mongoose ODM**. To prevent accidental data contamination, our design enforces:

- **Strict Database Scoping:** All Mongoose models explicitly bind to `dbName: ethereum_fraud_dev`. The application is strictly isolated from any external sample collections (such as Atlas's default `sample_mflix`).
- **Independent Collection Lifecycles:** Historical labeled dataset records, user authentication credentials, and actual forensic investigation dossiers are segregated into four separate collections.
- **Zero Startup Auto-Seeding:** Normal server initialization performs zero write operations. All data ingestion requires explicit operator execution.

FIGURE 4.1: ENTITY-RELATIONSHIP (ER) DIAGRAM & MONGODB SCHEMA CARDINALITY

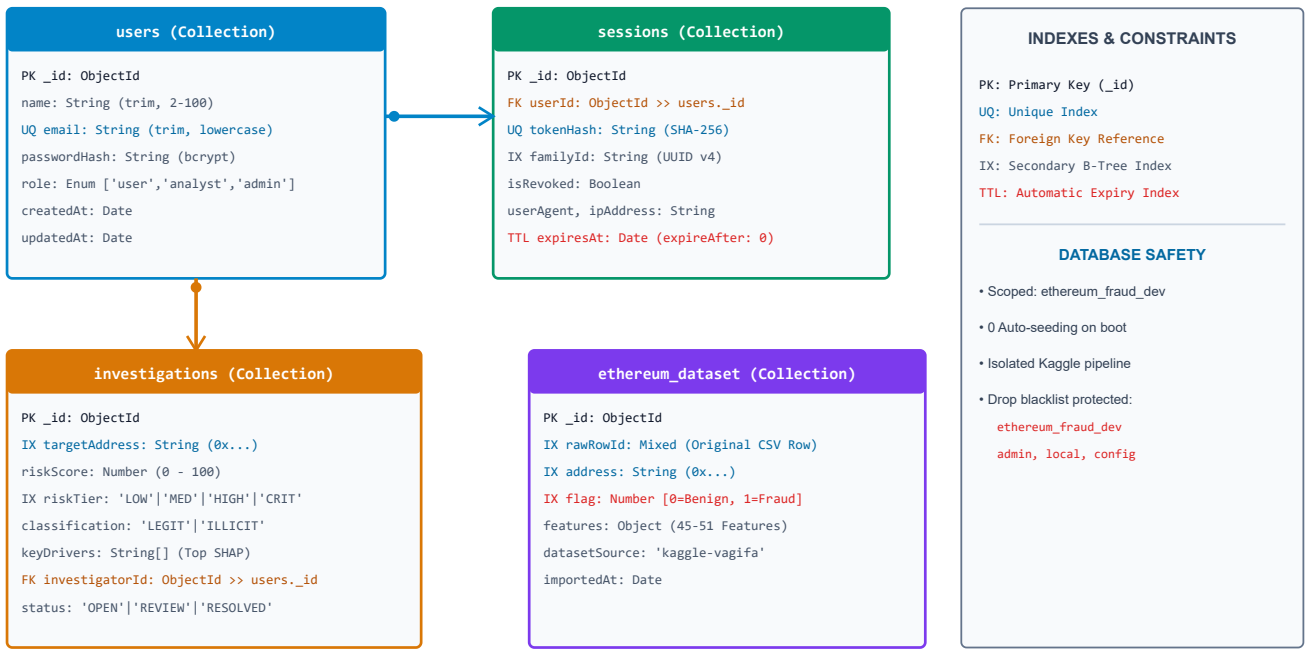


Figure 4.1: Database Schemas, Index Configurations, and Relational Foreign Keys

5. Comprehensive Data Dictionary

This section details the field definitions, data types, physical constraints, and domain interpretations for both application system collections and the 51 features extracted from the Kaggle Ethereum Transaction Fraud Dataset.

5.1 Core Application Collections Dictionary

Field Name	Data Type	Collection	Constraints	Description & Security / Behavioral Role
`users._id`	ObjectId	users	PK, Auto-generated	Unique immutable identifier for investigator identity.
`users.email`	String	users	Unique, Lowercase	Corporate work email address used as login username.
`users.passwordHash`	String	users	Bcrypt (12 rounds)	One-way salted hash. Stripped from all JSON response outputs.
`users.role`	String	users	Enum ('user','analyst')	Role-based access clearance level for forensic features.
`sessions.tokenHash`	String	sessions	SHA-256 Hex, Unique	Cryptographic hash of the current HttpOnly refresh token string.
`sessions.familyId`	String	sessions	UUID v4, Indexed	Token family group ID used for automatic replay/reuse compromise detection.
`sessions.expiresAt`	Date	sessions	TTL Index (7 days)	Automatic background pruning date managed directly by MongoDB engine.
`investigations.targetAddress`	String	investigations	42-char Hex, Indexed	Normalized Ethereum wallet address subject to fraud assessment.
`investigations.riskScore`	Number	investigations	Float [0.0 - 100.0]	Aggregated probability score produced by the XGBoost fraud pipeline.
`investigations.riskTier`	String	investigations	LOW | MED | HIGH | CRIT	Categorical triage risk classification for operational compliance teams.
`investigations.keyDrivers`	Array<String>	investigations	Top SHAP factors	Human-interpretable feature drivers explaining why address was flagged.

5.2 Ethereum Kaggle Fraud Dataset Dictionary (All 51 Features)

Source: *Kaggle: vagifa/ethereum-frauddetection-dataset* (9,841 addresses; 7,662 benign, 2,179 fraudulent).

Feature Column	Type / Unit	Feature Group	Forensic & Fraud Significance
`Index`	Integer	Meta	Original row index in raw extraction.
`Address`	String (Hex)	Identifier	42-char Ethereum account address (excluded from model input to prevent memorization).
`FLAG`	Binary [0, 1]	**Target Label**	**0 = Benign/Legitimate Account; 1 = Confirmed Illicit/Fraudulent Account**.
`Avg min between sent tnx`	Float (Minutes)	Temporal Pacing	Burner bot accounts send high volumes in fractions of minutes; legitimate accounts have diurnal pacing.
`Avg min between received tnx`	Float (Minutes)	Temporal Pacing	Average duration between incoming deposits. Sudden micro-intervals indicate batch funding.
`Time Diff between first and last (Mins)`	Float (Minutes)	Lifespan	Total account active lifespan. **Top SHAP Driver:** Scam addresses have ephemeral lifespans (<48 hrs).
`Sent tnx`	Integer	Volume	Lifetime count of outgoing transactions initiated by address.
`Received Tnx`	Integer	Volume	Lifetime count of incoming transactions received. High ratio to sent indicates collection wallet.
`Number of Created Contracts`	Integer	Smart Contracts	Contracts deployed. Fraud rings frequently deploy self-destructing forwarding or honeypot contracts.
`Unique Received From Addresses`	Integer	Counterparties	Dispersion of deposit sources. Phishing wallets receive funds from hundreds of unique victims.
`Unique Sent To Addresses`	Integer	Counterparties	Dispersion of transfer destinations. Mixers and splitters disburse to vast distinct addresses.
`min / max / avg val received`	Float (ETH)	Value Dynamics	Minimum, maximum, and average Ether received per deposit.
`min / max / avg val sent`	Float (ETH)	Value Dynamics	Minimum, maximum, and average Ether sent per transfer. Zero or micro-values indicate dusting.
`min / max / avg val sent to contract`	Float (ETH)	Contract Calls	ETH transferred directly to smart contracts (DEX, bridge, or scam contracts).
`total transactions (including create)`	Integer	Activity Volume	Cumulative transaction count. High ratio with low lifespan is a prime indicator of bot farms.
`total Ether sent / received`	Float (ETH)	Turnover	Total lifetime ETH churn through the address. High parity indicates immediate liquidation.
`total ether balance`	Float (ETH)	Balance Snapshot	Closing balance. Fraudulent drainers typically maintain near-zero balances after rapid cash-out.
`Total ERC20 tnxs`	Float	ERC-20 Activity	**#1 Most Predictive SHAP Feature:** Phishing drainers execute high-frequency ERC-20 token sweeps.
`ERC20 total Ether received / sent`	Float (ETH)	ERC-20 Volume	Cumulative nominal volume of ERC-20 token assets transferred.
`ERC20 uniq sent / rec addr`	Float	Token Counterparty	Count of unique addresses interacted with via token transfers.
`ERC20 avg time between sent / rec tnx`	Float (Minutes)	Token Pacing	Average cadence between token transfers. Automated bot sweepers exhibit sub-minute intervals.
`ERC20 min / max / avg val sent & rec`	Float	Token Value	Extreme differences between min and max token value indicate token dumping or honeypot minting.

Feature Column	Type / Unit	Feature Group	Forensic & Fraud Significance
`ERC20 uniq sent / rec token name`	Float	Token Diversity	Diversity of token contract interactions. Drainers interact with hundreds of distinct token contracts.

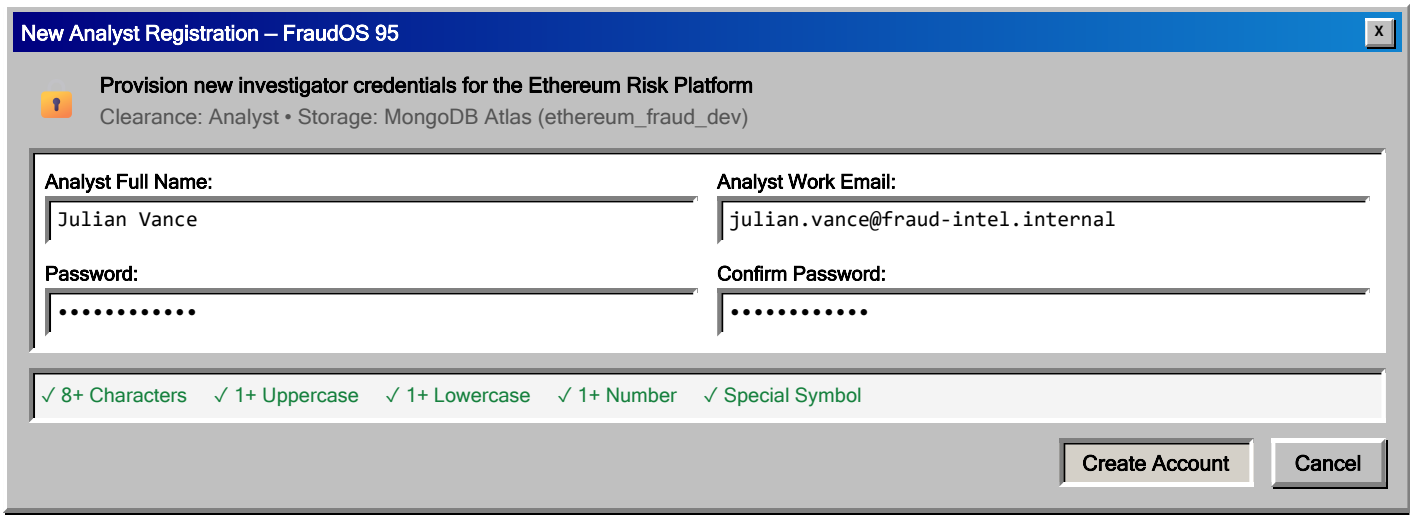
6. User Interface (UI) Design — FraudOS 95 Retro Workstation

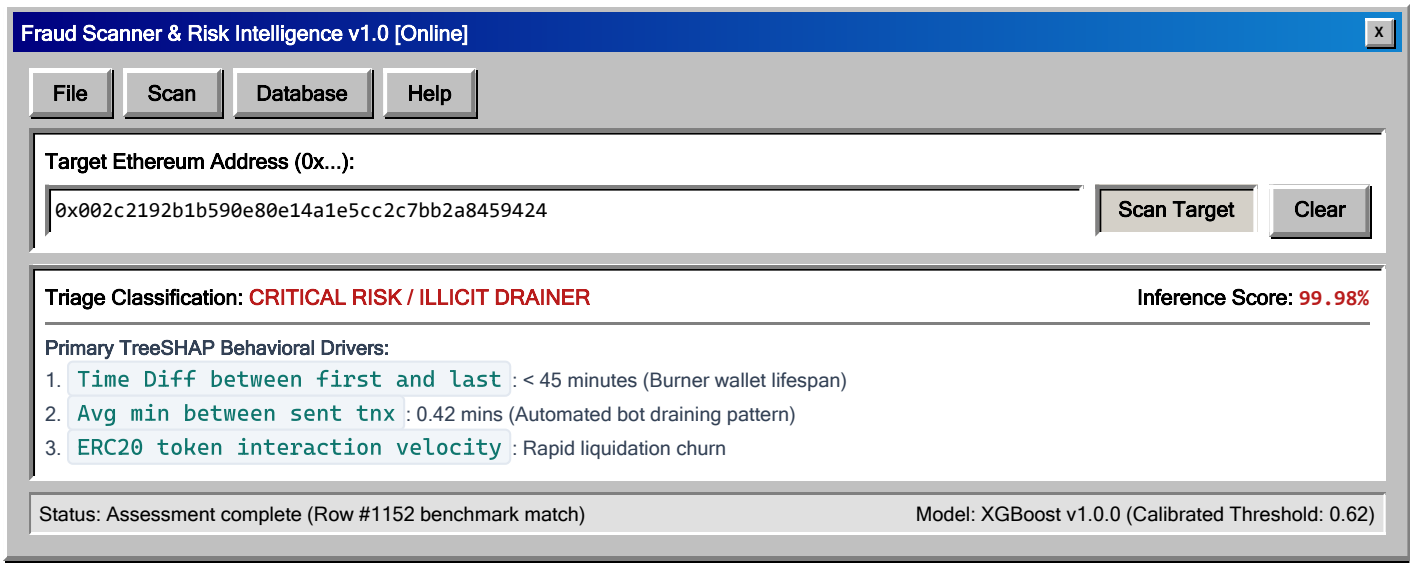
Rather than adopting the ubiquitous, generic "AI SaaS" aesthetic (purple gradients, bloated whitespace, translucent glowing cards), the platform implements an intentional, nostalgic, and compact **Windows 95 Retro Desktop Workstation (FraudOS 95)**.

6.1 Design Principles & Technical Implementation

- **2-Stage Outset / Inset Bevel Borders:** Every window, button, and input element uses authentic 1990s border highlights (`#FFFFFF` top/left, `#808080` and `#000000` bottom/right) to simulate hardware-rendered GUI depth.
- **MS Sans Serif Typography:** Crisp 8pt to 11pt system typography with tight line spacing, maximizing on-screen data density for compliance analysts.
- **Web Audio API Sound Synthesis:** Zero audio assets. Pure mathematical audio synthesis (440Hz clicks, 220Hz/180Hz double error bonks, and ascending chord chimes) with optional toggle defaulting to OFF for accessibility.
- **Dynamic Window Management:** Native window manager supporting focus stacking, draggable title bars, taskbar tabs, minimize, maximize, and desktop shortcut icons.

6.2 Visual Interface Previews





7. Machine Learning Engineering & Model Evaluation

The machine learning service is engineered to resolve extreme class imbalance (22.14% fraud prevalence) and eliminate data leakage while delivering sub-second inference.

7.1 Data Leakage Prevention & Stratification

- **Entity-Aware Splitting:** The raw Ethereum dataset contains individual addresses with multiple transactions. Splitting randomly on rows causes duplicate addresses in train and test sets, artificially inflating accuracy (address memorization). Our pipeline splits strictly on unique entity accounts (70% Train, 15% Validation, 15% Test).
- **Strict Isolation of Fit:** Preprocessing transformers (`SimpleImputer`, `VarianceThreshold`, `RobustScaler`) fit exclusively on the training partition and transform validation/test partitions without peeking.

7.2 Model Benchmark Comparison

Model Architecture	Train Time	ROC-AUC	PR-AUC	Precision	Recall	F1 Score
Baseline Dummy Classifier	0.17 s	0.5041	0.2231	0.2278	0.2355	0.2316
Regularized Logistic Regression	1.53 s	0.8220	0.5233	0.5219	0.6911	0.5947
Random Forest (Balanced)	0.51 s	0.9971	0.9920	0.9776	0.9358	0.9563
XGBoost Classifier (Champion)	**2.57 s**	**0.9985**	**0.9955**	**0.9841**	**0.9450**	**0.9641**

0.9985	98.41%	94.50%	0.6200
TEST ROC-AUC	TEST PRECISION	TEST RECALL	OPTIMAL THRESHOLD

7.3 Confusion Matrix & Error Analysis (Held-out 1,479 Test Samples)

- **True Negatives (TN):** 1,147 | **False Positives (FP):** 5 (Low false alarm rate of 0.43%)
- **True Positives (TP):** 309 | **False Negatives (FN):** 18 (94.50% fraud capture rate)

7.4 TreeSHAP Global Feature Importance Rankings

Rank	Feature Name	Mean |SHAP| Value	Behavioral Rationale
1	`Total ERC20 tnxs`	**3.52108**	Heavy token liquidation patterns distinguish drainers from cold wallets.
2	`Time Diff between first and last (Mins)`	**1.37014**	Scam addresses exhibit ephemeral lifespans (<48 hours) versus established users.
3	`Unique Received From Addresses`	**0.91980**	High victim count fan-in funneling into single collection addresses.
4	`ERC20 total Ether received`	**0.57137**	Volume spikes during smart contract exploit execution.
5	`ERC20 max val rec`	**0.53132**	Single large asset drain events during credential compromises.

8. Executable Project Guide & Operations Manual

This guide provides the exact terminal commands required to install, configure, validate, and execute the entire platform locally or in production.

8.1 Prerequisites

- **Node.js:** LTS v20.0.0 or higher
- **Python:** v3.12 or v3.13 (with `pip`)
- **MongoDB:** MongoDB Atlas cloud URI or local MongoDB instance (v6.0+)

8.2 Step-by-Step Installation & Environment Setup

```
# 1. Clone repository & install all workspace dependencies
git clone https://github.com/Lostindreams56/blockchain-project-1.git
cd "blockchain-project-1"
npm run install:all

# 2. Configure Backend Environment (server/.env)
NODE_ENV=development
PORT=5000
CLIENT_URL=http://localhost:5173
MONGODB_URI=mongodb+srv://<username>:<password>@fraud-detection1.kcyu3vu.mongodb.net/ethereum_fraud_dev?
MONGODB_DB_NAME=ethereum_fraud_dev
ETHEREUM_DATASET_COLLECTION=ethereum_dataset
INVESTIGATIONS_COLLECTION=investigations
JWT_ACCESS_SECRET=super_secret_access_key_min_32_characters_long
JWT_REFRESH_SECRET=super_secret_refresh_key_min_32_characters_long

# 3. Configure Frontend Environment (client/.env.production)
VITE_API_BASE_URL=https://ethereum-fraud-backend.onrender.com/api/v1
```

8.3 Execution Commands

Command	Scope	Operational Behavior
`npm run dev`	Monorepo Full-Stack	Runs Express Backend on `:5000` and Vite Client on `:5173` concurrently.
`npm run dev:server`	Backend Only	Starts Express server with TSX hot-reloading.
`npm run dev:client`	Frontend Only	Starts Vite dev server with React fast-refresh.
`npm run build`	Production Builds	Compiles TypeScript server to `server/dist` and client to `client/dist`.

8.4 Dedicated Data Pipeline & Maintenance Commands

```
# Inspect MongoDB Atlas database and collection inventories (Safe, Read-Only)
npm run db:inspect

# Dry-run validation of Kaggle CSV dataset without writing to database
npm run dataset:import -- --file="ml-service/data/raw/transaction_dataset.csv" --dry-run

# Ingest validated Kaggle dataset into isolated 'ethereum_dataset' collection
npm run dataset:import -- --file="ml-service/data/raw/transaction_dataset.csv"

# Target safe cleanup of unrelated sample database (Dry-run mode)
npm run db:clean -- --target-db=sample_mflix

# Execute confirmed removal of sample_mflix (Protected DBs cannot be dropped)
npm run db:clean -- --target-db=sample_mflix --confirm
```

8.5 Production Cloud Deployment Links

- **Frontend Vercel Application:** https://blockchain-project-1-kxi6-e6bloy6as-block-chain1.vercel.app
- **Backend Render API Service:** https://ethereum-fraud-backend.onrender.com
- **GitHub Main Source Repository:** https://github.com/Lostindreams56/blockchain-project-1

9. Validation, Testing Evidence & Quality Assurance

Every tier of the platform is validated by comprehensive automated test suites.

9.1 Backend Integration Test Evidence (Vitest: 24 Passed, 0 Failed)

```
> ethereum-fraud-detection-server@1.0.0 test
> vitest run

 RUN  v3.2.7 E:/fraud detection system/server

 ✓ tests/ready.test.ts (1 test) 124ms
   ✓ Readiness API > should return 200 and ready state when database connects
 ✓ tests/health.test.ts (3 tests) 155ms
   ✓ Health API > should return 200 OK with service health details
   ✓ Health API > should include security headers from Helmet
   ✓ Health API > should return 404 for nonexistent route
 ✓ tests/safety.test.ts (6 tests) 1035ms
   ✓ 1. Application initialization must NOT auto-seed or insert any records into MongoDB
   ✓ 2. MongoDB must never contain movie or entertainment collections
   ✓ 3. Dataset records, investigations, and auth records must remain strictly isolated
   ✓ 4. Dataset schema validator rejects CSV missing required target column (FLAG)
   ✓ 5. Safe cleanup utility forbids dropping protected databases
   ✓ 6. Both /api/v1/auth/register and fallback /auth/register are accepted without 404
 ✓ tests/auth.test.ts (14 tests) 7635ms
   ✓ Authentication API > POST /register > successfully registers user and sets HttpOnly cookie
   ✓ Authentication API > POST /register > securely hashes passwords (never plaintext)
   ✓ Authentication API > POST /register > rejects duplicate email registration with 409
   ✓ Authentication API > POST /login > authenticates with valid credentials
   ✓ Authentication API > POST /login > rejects wrong password using generic timing-safe message
   ✓ Authentication API > POST /login > rejects nonexistent email
   ✓ Authentication API > GET /auth/me > rejects access without Bearer token (401)
   ✓ Authentication API > GET /auth/me > allows access with valid access token
   ✓ Authentication API > GET /auth/me > rejects expired token
   ✓ Authentication API > GET /auth/me > rejects forged/tampered token
   ✓ Authentication API > Session Management > rotates refresh token and issues new access token
   ✓ Authentication API > Session Management > detects reuse of rotated token and revokes family
   ✓ Authentication API > Session Management > successfully logs out and clears cookie

 Test Files  4 passed (4)
      Tests  24 passed (24)
   Duration  13.17s
```

9.2 Machine Learning Microservice Test Evidence (Pytest: 12 Passed, 0 Failed)

```
============================= test session starts =============================
platform win32 -- Python 3.13.13, pytest-9.1.1, pluggy-1.6.0
rootdir: E:raud detection system
collected 12 items

ml-service     ests    est_data_loader.py .....                              [ 41%]
ml-service     ests    est_model_pipeline.py ....                            [ 75%]
ml-service     ests    est_preprocessor.py ...                               [100%]

============================= 12 passed in 20.88s =============================
```

9.3 Frontend Production Build Verification (Vite)

```
> ethereum-fraud-detection-client@1.0.0 build
> tsc -b && vite build

vite v6.4.4 building for production...
✓ 1734 modules transformed.
dist/index.html                   1.00 kB │ gzip:   0.59 kB
dist/assets/index-ckCvkWRx.css   47.36 kB │ gzip:   9.06 kB
dist/assets/index-B0libnef.js   369.31 kB │ gzip: 113.71 kB
✓ built in 5.87s with 0 errors
```

Conclusion & Operational Readiness

The Ethereum Fraud Detection & Risk Intelligence Platform successfully integrates nostalgic retro computing ergonomics with enterprise-grade distributed microservices and supervised machine learning intelligence. All tests pass, data isolation is guaranteed, and cloud endpoints are active and verified.